INSTITUTE CO{}ELINE

Introduction to AI and Tools

DAY 1 — PRACTICAL TASKS WORKBOOK

Session: 4 Hours | 12 Tasks | Individual & Pair Exercises

Complete each task using the AI tool of your choice. Write your prompts and observations in the spaces provided. Tasks alternate between general business scenarios and software development contexts.

Participant Name:	Rahaf Alkiyumi	**Date:**	12/4/2026
Job Title:		**Group:**	

■ Time Block	Section	Tasks	Focus Area
0:00 – 0:40	A — AI Understanding	1–2	AI concepts, tools overview, responsible use
0:40 – 1:30	B — Prompt Engineering	3–5	CRTFC framework, prompt quality, advanced techniques
1:30 – 2:30	C — Communication & Docs	6–8	Emails, documentation, meeting summaries
2:30 – 3:20	D — Data & Code Tasks	9–10	Document analysis, code-related AI tasks
3:20 – 4:00	E — Workflow Design	11–12	Automation templates, personal AI workflow plan

■ Blue tasks = General business context	■■■ Purple tasks = Software development context	■ Mixed = Both perspectives combined

SECTION A | AI Concepts, Tools & Responsible Use ■ 0:00 – 0:40

Complete Tasks 1–2 in this section. Start from memory, then verify using your AI tool. The goal is to build a solid conceptual foundation before applying AI to real tasks.

Task 1 AI Knowledge Audit & Tool Exploration ■ 20 min ● Beginner

■ **Instructions:**

1. Without using AI, write your own definition of AI and list what you think it can and cannot do (use the table below).

2. Now open ChatGPT or Claude. Ask: 'Explain what a Large Language Model is in simple terms for a software developer who has never used AI tools before.'

3. Compare the AI's explanation to yours. Note 1 concept that was new or surprising to you.

4. Try a second tool (e.g., if you used ChatGPT, try Claude or Gemini with the exact same prompt). Note any differences in the responses.

5. Ask one of the tools: 'What are the top 5 use cases for AI tools in software development?' Note its answer in the space provided.

■ **Deliverable:** Completed tables + 2 tool comparison notes + developer use cases listed.

My Definition of AI (before AI tool)	AI's Explanation (from the tool)
is the capacity of machines or computers to carry out operations that often call for human intelligence.	is the field focused on creating machines or software that can perform tasks that typically require human intelligence, such as perception, reasoning, learning, decision-making, and language understanding.

■ AI CAN (your view)	■■ AI CANNOT / Risks (your view)
1. Data Processing	1. Lack of True Understanding
2. Prediction	2. Depends on Data Quality
3. Natural Language Processing	3. Limited Creativity
4. Pattern Recognition	4. No Emotions or Consciousness

■ *Difference I noticed between Tool 1 and Tool 2:*

Write your answer / prompt here:

AI is the ability of machines to learn from data and do human-like functions, such as thinking, responding, and solving problems.

LLM: An AI that analyzes a lot of text and makes word predictions to provide responses that resemble those of a human.

■■■ *Top 5 AI use cases for software development (from AI's answer):*

Write your answer / prompt here:

1- AI-assisted coding and code completion

2- Automated testing, test generation, and defect detection

3- DevOps and software delivery optimization

4- Codebase analysis, modernization, and refactoring

5- Natural language interfaces for software tools and documentation

Task 2 Responsible AI — Risk Scenarios for Developers

■ 20 min ● Beginner

■ **Instructions:**

1. Read each scenario. Mark SAFE ■ or RISKY ■■ and write a one-sentence reason.

2. For the RISKY ones, write an alternative approach that achieves the same goal safely.

3. After completing your assessment, ask ChatGPT: 'What are the top 5 AI data privacy risks for software developers?' Compare its answer to your assessment.

■ **Deliverable:** Completed risk table + alternative safe approaches written.

Scenario	Safe / Risky?	Reason / Safe Alternative
You paste a function from your company's payment processing module to get a code review	■■	It may share private banking systems or corporate logic. Distribute a dummy or simplified version of the function that lacks actual logic or secrets.
You ask AI: 'Explain how JWT token refresh logic works' without pasting any code	■	No sensitive data is shared
You paste your production server's nginx config file including the server IP and domain names	■■	reveals potentially exploitable infrastructural details Before sharing, remove IPs and domains and replace them with placeholders.
You describe a bug scenario: 'I have a function that processes user objects — it crashes when email is null'	■	General description with no sensitive data
You upload a CSV of customer order history (with names, emails, and order amounts) to ask for analysis	■■	includes sensitive and private client information Utilize fictitious or anonymised data without actual user information
You ask: 'Write a Python script to parse CSV files with columns: order_id, amount, status'	■	No real data or secrets involved
You paste your AWS .env file to ask AI why your connection string isn't working	■■	includes data and credentials (passwords, API keys). Before sharing, replace all secrets with dummy values

You ask AI to suggest database index strategies for a table with 10 million rows	[redacted]	only general performance optimization is shared; No private information

SECTION B | Prompt Engineering Practice ■ 0:40 – 1:30

Tasks 3–5 build your prompt writing skills from basic to advanced. Use the CRTFC framework. Test every prompt in your AI tool and refine based on results.

Task 3 Spot and Fix Weak Prompts ■ 15 min ● Beginner

■ **Instructions:**

1. Each prompt below is weak. Identify WHY it is weak, then rewrite it using the CRTFC framework.
2. Test your improved version in the AI tool. Rate whether it gave a significantly better result.

■ **Deliverable:** 3 improved prompts written + result quality ratings.

■ Weak Prompt 1 (General): *"Write documentation"*

Why is it weak? It's too vague—no context, purpose, or format is given.

CONTEXT	*Write your content here...*	I'm a PM working on a 6-month ERP integration project.
ROLE	*Write your content here...*	You are a technical writer.
TASK	*Write your content here...*	Write clear API documentation explaining endpoints, request/response, and usage.
FORMAT	*Write your content here...*	Use headings, bullet points, and code examples.
CONSTRAINTS	*Write your content here...*	Keep it concise and beginner-friendly.

Result quality: ■ Much better ■ Slightly better ■ No improvement

■■■ Weak Prompt 2 (Dev): *"Help me with my code"*

Why is it weak? No details about the code, problem, or expected outcome.

CONTEXT	*Write your content here...*	I'm a backend developer working on a FinTech API in Python.
ROLE	*Write your content here...*	You are a senior software engineer.
TASK	*Write your content here...*	Identify the issue and suggest a fix.
FORMAT	*Write your content here...*	Explain the problem, then provide corrected code.
CONSTRAINTS	*Write your content here...*	Keep explanation simple and avoid unnecessary complexity.

Result quality: ■ Much better ■ Slightly better ■ No improvement

■ Weak Prompt 3 (General): *"Make a summary"*

Why is it weak? It doesn't specify what to summarize, how long the summary should be, or the style.

CONTEXT	*Write your content here...*	I'm a student summarizing a 2-page article about AI and jobs.

ROLE	*Write your content here...*	You are a professional content writer.
TASK	*Write your content here...*	Summarize the key points of the article.
FORMAT	*Write your content here...*	Write a short paragraph with 4–5 sentences.
CONSTRAINTS	*Write your content here...*	Keep it simple, clear, and avoid technical jargon.

Result quality: ■ *Much better* ■ *Slightly better* ■ *No improvement*

■■■ Weak Prompt 4 (Dev): *"Fix the bug"*

Why is it weak? No details about the bug, code, error message, or expected behavior.

CONTEXT	*Write your content here...*	I'm a backend developer working on a Node.js API where a function fails when handling empty input
ROLE	*Write your content here...*	Act as a senior backend developer and debugger.
TASK	*Write your content here...*	Identify the bug and provide a fix for the function.
FORMAT	*Write your content here...*	Explain the issue, then show the corrected code.
CONSTRAINTS	*Write your content here...*	Keep the fix simple, efficient, and secure.

Result quality: ■ *Much better* ■ *Slightly better* ■ *No improvement*

■ Weak Prompt 5 (General): *"Write an email about the meeting"*

Why is it weak? It doesn't specify the meeting purpose, audience, tone, or key details.

CONTEXT	*Write your content here...*	I'm a project coordinator scheduling a weekly team meeting for a software development project.
ROLE	*Write your content here...*	Act as a professional business communication assistant.
TASK	*Write your content here...*	Write a formal email informing the team about the meeting details.
FORMAT	*Write your content here...*	Email format with subject line, greeting, body, and closing.
CONSTRAINTS	*Write your content here...*	Keep it clear, professional, and concise. Mention time, purpose, and agenda placeholder.

Result quality: ■ *Much better* ■ *Slightly better* ■ *No improvement*

Task 4 Build a Level-4 Expert Prompt

■ 20 min ● Intermediate

■ **Instructions:**

1. Choose ONE scenario below (circle your choice).

2. Scenario A (General): You are a project manager. Write a status email to the steering committee summarizing milestone progress, a risk, and the next sprint goal.

3. Scenario B (Dev): You are a backend developer. Ask AI to review a Python function that processes payment transactions for correctness, security, and edge cases.

4. Scenario C (Dev): You are a tech lead. Ask AI to compare three caching strategies (Redis, Memcached, in-memory) for a high-traffic API with specific tradeoffs.

5. Fill in the CRTFC table below, then write your full prompt in a single paragraph.

6. Test the prompt. Rate the result. Then write one follow-up prompt to refine the output.

■ **Deliverable:** Full CRTFC prompt + follow-up refinement prompt + quality rating.

I chose Scenario: B (Dev)

CONTEXT	*Describe your role, company, and situation...*	I'm a backend developer working on a FinTech system that processes online payment transactions in Python.
ROLE	*Act as a [type of professional/expert]...*	Act as a senior backend engineer and security-focused code reviewer.
TASK	*Write / Draft / Review / Analyze / Generate...*	Review the Python function for correctness, security issues, and edge cases. Identify bugs and suggest improvements.
FORMAT	*Specify structure, length, tone, output format...*	Provide a structured response with: - Issues found - Security concerns - Edge cases -Improved code
CONSTRAINTS	*What to avoid, required elements, version, audience...*	Focus on production-level quality. Ensure secure handling of financial data. Avoid overly complex explanations.

■ *My Full Prompt (combine all CRTFC elements):*
Write your answer / prompt here:

Context: I'm a backend developer working on a FinTech Python system handling payment transactions (validation, balance checks, execution). Role: Act as a senior backend engineer and security reviewer. Task: Review the function for bugs, security issues, and edge cases, and suggest fixes. Format: List issues, security risks, edge cases, then improved code if needed. Constraints: Keep it concise, production-focused, and easy to understand.

■ *Follow-up refinement prompt (what did you ask next to improve the output?):*
Write your answer / prompt here:

Refine your previous answer by adding specific real-world attack scenarios (e.g., double spending, race conditions, API replay attacks) and explain how each improvement prevents them. Also simplify the explanation for junior developers.

Initial result quality:	■ Excellent ■ Good ■ Needs work	After refinement:	■ Excellent ■ Good ■ Needs work

Task 5 Advanced Prompting Techniques ■ 15 min ● Advanced

■ **Instructions:**

1. Practice each advanced technique using the instructions below.

2. TECHNIQUE A — Chain of Thought: Ask AI: 'Explain step by step how a JWT token is validated on an API server. Then identify 3 potential security vulnerabilities in a basic implementation.' Paste the response summary below.

3. TECHNIQUE B — Few-Shot Prompting: Write a prompt that gives the AI 2 examples of a code comment format you want, then asks it to apply that format to a new function.

4. TECHNIQUE C — Iterative Refinement: Start with a simple prompt, then write 2 follow-up prompts that progressively refine the output (e.g., add error handling, then add logging, then add a unit test).

■ **Deliverable:** 3 technique examples attempted + observations noted.

Technique A — Chain of Thought

■ *Write your prompt(s) here:*

- The API extracts the JWT from the request (usually from the Authorization header).
- It splits the token into header, payload, and signature.
- The server verifies the signature using a secret key or public key.
- It checks token claims like expiration (exp), issuer (iss), and audience (aud).
- If valid, the request is authenticated; otherwise, it is rejected.

Technique B — Few-Shot Prompting

■■■ *Write your prompt(s) here:*

Example 1–
# Function: add_numbers
# Purpose: Adds two numbers and returns the result
def add_numbers(a, b):
return a + b

Example 2—
# Function: greet_user
# Purpose: Returns a greeting message for a user
def greet_user(name):
return f"Hello {name}"

Now apply the same comment format to this function:

def multiply(x, y):
return x * y

Technique C — Iterative Refinement (show all 3 prompts)

■ *Write your prompt(s) here:*

Prompt 1 (Basic):

Write a Python function that divides two numbers.

Prompt 2 (Refinement — Error Handling):

Update the function to handle division by zero and invalid inputs.

Prompt 3 (Refinement — Logging + Test):

Enhance the function by adding logging for errors and write a simple unit test for it.

SECTION C | AI for Communication & Documentation ■ 1:30 – 2:30

Tasks 6–8 apply AI to real communication and documentation scenarios. All prompts should use the CRTFC framework. Save your best prompts — you will reuse them in Section E.

Task 6 Professional Email Writing — General & Developer Scenarios ■ 20 min ● Intermediate

■ **Instructions:**

1. Write and test a professional email for BOTH scenarios below (one general, one developer-specific).

2. SCENARIO 1 (General): You are a product manager. Write an email to senior management requesting approval for a 2-week delay in the product launch due to critical QA findings. Keep it under 200 words, professional, and include a revised timeline.

3. SCENARIO 2 (Dev): You are a backend developer. Write an email to the DevOps team requesting emergency access to production logs to investigate a P1 incident impacting 15% of users. Include: the affected service, symptoms, and what you need access to.

4. For each: fill in the CRTFC table, write the full prompt, test it, and rate the result.

5. BONUS: Ask AI to rewrite your best email in a different tone (more urgent / more formal / more concise).

■ **Deliverable:** 2 complete email prompts + both emails reviewed and rated.

■ Scenario 1: General — Launch Delay Approval

CONTEXT	You are a Product Manager overseeing the launch of a customer-facing digital product. During final QA, critical issues were discovered that could negatively impact user experience and system stability if released on schedule.
ROLE	Act as a senior Product Manager communicating with executive leadership.
TASK	Request approval for a 2-week launch delay.
FORMAT	Subject……Greeting……Clear justification ….Revised timeline……Closing
CONSTRAINTS	Under 200 words…….Professional and persuasive……..Concise and clear

Full prompt:
Write your answer / prompt here:

Context:
You are a Product Manager overseeing a product launch. Critical QA issues have been identified that may impact user experience and stability.

Role:
Act as a senior Product Manager communicating with executive leadership.

Task:
Write an email requesting approval for a 2-week delay in the product launch.

Format:
Include subject, greeting, justification, revised timeline, and closing.

Constraints:
Keep it under 200 words, professional, concise, and persuasive.

■ Email is professional & ready to send ■ Needs minor edits ■ Needs major revision

■■■ Scenario 2: Dev — Emergency Production Access

CONTEXT	You are a backend developer working on a FinTech platform that processes real-time payment transactions in Python. A P1 production incident is impacting ~15% of users, causing failed or delayed transactions.

ROLE	Act as a senior backend engineer and security-focused incident responder.
TASK	Request urgent access to production logs.
FORMAT	Subject….Greeting…..Incident summary…..Affected service….Symptoms….Access request…..Closing
CONSTRAINTS	Clear, urgent, professional, Technically precise, Concise

Full prompt:

Write your answer / prompt here:

Context:
You are a backend developer working on a FinTech platform that processes real-time payment transactions in Python. A P1 incident is impacting ~15% of users, causing failed or delayed transactions.

Role:
Act as a senior backend engineer and security-focused incident responder.

Task:
Write an urgent email to the DevOps team requesting access to production logs.

Format:
Subject, greeting, concise explanation, request, closing.

Constraints:
Keep it concise, professional, and urgent.

■ Email is professional & ready to send ■ Needs minor edits ■ Needs major revision

■ *BONUS — Tone Variation (paste the AI's alternative version below):*

Write your answer / prompt here: Rewrite the email in a more formal and executive tone.

Dear DevOps Team,

We are currently addressing a high-priority (P1) incident affecting approximately 15% of users within the Payment Processing Service.

The issue is presenting as failed and delayed transactions, increased latency, and intermittent API errors, with potential implications for transaction reliability.

To proceed with root cause analysis, I kindly request expedited access to the following:

- Production logs (past 24 hours)
- Error logs and trace data
- Recent deployment and infrastructure updates

Your prompt assistance is essential to mitigate impact and restore service stability.

Thank you for your cooperation.

Kind regards,
[Your Name]

Task 7 Technical Documentation Generation ■ 25 min ● Intermediate

■ Instructions:

1. Practice generating three types of technical documentation using AI.

2. DOC TYPE 1 — Docstring: Write a prompt asking AI to add a Google-style docstring to the sample Python function provided below.

3. DOC TYPE 2 — README Section: Write a prompt to generate an 'Installation & Setup' README section for a Node.js REST API. Include: prerequisites, clone, install dependencies, configure .env, run dev server.

4. DOC TYPE 3 — Commit Message: Write a prompt asking AI to generate a conventional commit message for these changes: 'Added Redis caching layer to the /products endpoint, reducing average response time from 480ms to 35ms. Also fixed a null pointer bug when product category is undefined.'

5. Test all three prompts. Note which output required the least editing.

■ **Deliverable:** 3 documentation prompts written + one 'least editing needed' observation.

```
# Sample function for documentation exercise: def process_user_order(user_id, items,
discount_code=None): total = sum(item['price'] * item['qty'] for item in items) if
discount_code: discount = get_discount(discount_code) total = total * (1 - discount) if
total < 0: raise ValueError('Total cannot be negative') return {'user_id': user_id,
'total': round(total, 2), 'items_count': len(items)}
```

Doc Type 1 — Docstring Prompt:

Write your answer / prompt here:

Add a Google-style Python docstring to the following function. Include:

- Description of what the function does
- Arguments with types and descriptions
- Return value with type and description
- Possible exceptions raised

```
def process_user_order(user_id, items, discount_code=None):
    total = sum(item['price'] * item['qty'] for item in items)
    if discount_code:
        discount = get_discount(discount_code)
        total = total * (1 - discount)
    if total < 0:
        raise ValueError('Total cannot be negative')
    return {'user_id': user_id, 'total': round(total, 2), 'items_count': len(items)}
```

Doc Type 2 — README Section Prompt:

Write your answer / prompt here:

Generate a professional README “Installation & Setup” section for a Node.js REST API project.

Include the following sections clearly:

- Prerequisites (Node.js, npm, etc.)
- Clone the repository
- Install dependencies
- Configure environment variables using a .env file
- Run the development server

Format it in clean Markdown with code blocks and commands.

Doc Type 3 — Commit Message Prompt:

Write your answer / prompt here:

Generate a Conventional Commit message for the following changes:

- Added Redis caching layer to the /products endpoint, reducing average response time from 480ms to 35ms
- Fixed a null pointer bug when product category is undefined

Follow Conventional Commits format (type, scope, subject, body). Keep it concise but informative.

Which doc type required the least editing? ■ Docstring ■ README ■ Commit Message

Why? The structure of Conventional Commits is strict and predictable, making it easy for AI to follow.

Task 8 Meeting Notes & Report Generation ■ 15 min ● Intermediate

■ Instructions:

1. Read the raw meeting notes below.

2. Write a prompt asking AI to organize these notes into a professional meeting summary with sections: Attendees, Key Decisions, Action Items (with owner and deadline), and Blockers.

3. Write a second prompt asking AI to convert the same notes into a 5-slide presentation outline.

4. Compare both outputs and note which format is most useful for different audiences.

■ **Deliverable:** 2 prompts written + audience suitability comparison noted.

```
MEETING NOTES — Sprint Planning — March 4 Present: Ahmed (backend), Sara (frontend), Khalid
(DevOps), Nour (PM) Ahmed - auth service refactor done, PR open, needs review by EOD
tomorrow Sara - dashboard blocked waiting for API spec from Ahmed Khalid - deployment
pipeline broken on staging, no ETA yet, investigating Agreed - launch date pushed from
March 10 to March 17 because of pipeline issue Nour - needs to update client on delay -
email by tomorrow morning Ahmed to review Sara's frontend PR before end of week Khalid to
give update on pipeline at daily standup Friday Next sprint planning March 11 at 10am
```

■ *Prompt 1 — Meeting Summary:*
Write your answer / prompt here:

Convert the following raw meeting notes into a professional meeting summary.

Structure the output using these sections:

- Attendees
- Key Decisions
- Action Items (include owner and deadline)
- Blockers

Ensure the tone is clear, concise, and suitable for internal team documentation.

Meeting Notes///

Sprint Planning — March 4
Present: Ahmed (backend), Sara (frontend), Khalid (DevOps), Nour (PM)
Ahmed - auth service refactor done, PR open, needs review by EOD tomorrow
Sara - dashboard blocked waiting for API spec from Ahmed
Khalid - deployment pipeline broken on staging, no ETA yet, investigating
Agreed - launch date pushed from March 10 to March 17 because of pipeline issue
Nour - needs to update client on delay email by tomorrow morning
Ahmed to review Sara's frontend PR before end of week
Khalid to give update on pipeline at daily standup Friday
Next sprint planning March 11 at 10am

■ *Prompt 2 — Presentation Outline:*
Write your answer / prompt here:

Convert the following meeting notes into a 5-slide presentation outline.

Requirements:

- Each slide should have a title and bullet points
- Keep content concise and presentation-ready
- Slides should cover: overview, progress updates, issues/blockers, decisions, and next steps

Meeting Notes:

Sprint Planning — March 4
Present: Ahmed (backend), Sara (frontend), Khalid (DevOps), Nour (PM)
Ahmed - auth service refactor done, PR open, needs review by EOD tomorrow
Sara - dashboard blocked waiting for API spec from Ahmed
Khalid - deployment pipeline broken on staging, no ETA yet, investigating
Agreed - launch date pushed from March 10 to March 17 because of pipeline issue
Nour - needs to update client on delay email by tomorrow morning
Ahmed to review Sara's frontend PR before end of week
Khalid to give update on pipeline at daily standup Friday
Next sprint planning March 11 at 10am

Meeting summary is more useful for: ■ Developers ■ Management ■ All Presentation outline is more useful for: ■ Developers ■ Management ■ Client meetings

- Provides detailed, structured information
- Clearly tracks responsibilities, deadlines, and blockers
- Useful for documentation, follow-ups, and accountability

SECTION D | AI for Data Analysis & Code Tasks ■ 2:30 – 3:20

Tasks 9–10 focus on two of the highest-value AI use cases for developers: data and document analysis, and code-related tasks including review, debugging, and test generation.

Task 9 Document Analysis & Data Extraction ■ 25 min ● Intermediate

■ **Instructions:**

1. Use the sample sprint report below for all prompts in this task.
2. PROMPT 1: Write a prompt to summarize the report in exactly 5 bullet points (max 15 words each).
3. PROMPT 2: Write a prompt to extract all action items as a table with columns: Action | Owner | Deadline | Priority.
4. PROMPT 3: Write a prompt to convert the 'Performance Metrics' section into a structured JSON object.
5. PROMPT 4 (Advanced): Write a prompt asking AI to identify risks mentioned in the report and suggest mitigations.
6. Test all 4 prompts. Rate each for output quality and immediate usability.

■ **Deliverable:** 4 prompts written + quality ratings + JSON output from Prompt 3 pasted.

```
SPRINT 14 PERFORMANCE REPORT — Week of March 3 COMPLETED: - User authentication service refactored (owner: Ahmed, 3 days ahead of schedule) - Payment gateway integration tested — all 47 test cases passing - Mobile app API endpoints v2 deployed to staging IN PROGRESS: - Customer dashboard frontend (Sara, 60% complete, blocked on API spec) - CI/CD pipeline fix for staging environment (Khalid, ETA: March 7) BLOCKERS: - Staging deployment broken since March 2 — no prod deployments possible until resolved - API spec for dashboard not finalized — Sara blocked for 2+ days PERFORMANCE METRICS: - Sprint velocity: 42 story points (target: 45, -6.7%) - Bug count opened this sprint: 7 (4 resolved, 3 open) - Code review turnaround: avg 18 hours (target: 24 hours) ■ - Test coverage: 74% (target: 80%, gap: -6%) ACTION ITEMS: - Ahmed: Share API spec with Sara by March 5 - Khalid: Resolve pipeline issue by March 7 - Nour: Notify client of revised launch date by March 5 - All: Increase test coverage to 80% before next sprint review
```

Prompt 1 — 5-Bullet Summary:

Write your answer / prompt here:

- Authentication refactor completed early; payment integration fully tested; APIs deployed to staging
- Dashboard frontend 60% complete, blocked by missing API specification
- Staging deployment failure blocks production releases since March 2
- Sprint velocity slightly below target; test coverage and bugs need improvement
- Action items focus on API delivery, pipeline fix, client update, and test coverage increase

■ Ready to use ■ Needs minor edits ■ Needs major revision

Prompt 2 — Action Items Table:

Write your answer / prompt here:

Action	Owner	Deadline	Priority
Share API spec with Sara	Ahmed	March 5	High
Resolve pipeline issue	Khalid	March 7	High
Notify client of revised launch date	Nour	March 5	High
Increase test coverage to 80%	All	Not specified	Medium

■ Ready to use ■ Needs minor edits ■ Needs major revision

Prompt 3 — JSON Conversion:

Write your answer / prompt here:


```
{
  "performance_metrics": {
    "sprint_velocity": {
      "actual": 42,
      "target": 45,
      "difference_percentage": -6.7
    },
    "bug_count": {
      "opened": 7,
      "resolved": 4,
      "open": 3
    },
    "code_review_turnaround_hours": {
      "actual_average": 18,
      "target": 24,
      "difference_hours": -6
    },
    "test_coverage": {
      "actual_percentage": 74,
      "target_percentage": 80,
      "gap_percentage": -6
    }
  }
}
```


■ Ready to use ■ Needs minor edits ■ Needs major revision

Prompt 4 — Risk Analysis:

Write your answer / prompt here:

Risk 1: Staging Deployment Failure

- Impact: Blocks all production deployments → delays release timeline
- Mitigation: Prioritize pipeline fix, allocate additional DevOps resources, create fallback deployment plan

———

Risk 2: Missing API Specification

- Impact: Blocks frontend progress → delays feature completion
- Mitigation: Fast-track API spec delivery, enable parallel mock API usage

Risk 3: Below-Target Sprint Velocity

- Impact: Reduced delivery capacity → risk of sprint spillover
- Mitigation: Reassess workload, improve estimation accuracy, remove blockers early

———

Risk 4: Low Test Coverage (74%)

- Impact: Increased risk of bugs in production
- Mitigation: Enforce test coverage thresholds, prioritize automated tests

———

Risk 5: Open Bugs Remaining

- Impact: Potential quality issues at release
- Mitigation: Prioritize bug resolution before next sprint review

■ Ready to use ■ Needs minor edits ■ Needs major revision

■ *JSON output from Prompt 3 (paste or describe it):*

- AI gives solid but slightly generic mitigations
- In real work, you might refine to:
- Specific tools (e.g., Jenkins fix, API mocking tool)
- Exact owners
- Clear deadlines

Task 10 AI-Assisted Code Review, Debugging & Test Generation

■ 25 min ● Advanced

■ Instructions:

1. Complete the 3 code-related sub-tasks below using the sample code provided.

2. SUB-TASK A — Code Review: Write a CRTFC prompt asking AI to review the Python function below for: correctness, edge cases, security, and PEP8 compliance. Then test it.

3. SUB-TASK B — Bug Investigation: The function below has a logic bug. Write a prompt asking AI to identify the bug and provide a fixed version with an explanation.

4. SUB-TASK C — Test Generation: Write a prompt asking AI to generate pytest unit tests for the fixed version of the function, covering: happy path, empty input, null values, and boundary conditions.

5. REFLECTION: Which sub-task produced the most impressive AI output? Note what surprised you.

■ **Deliverable:** 3 CRTFC prompts + all results noted + reflection written.

```
# Code for review — find the bug and write tests: def calculate_discount(cart_items,
promo_code): """Apply discount to cart total."""  PROMO_CODES = { 'SAVE10': 0.10,
'SAVE20': 0.20, 'VIP50': 0.50 } total = 0 for item in cart_items: total += item['price']
* item['quantity'] if promo_code in PROMO_CODES: discount = PROMO_CODES[promo_code] total
= total - discount # BUG IS HERE if total < 0: total = 0 return total
```

Sub-Task A — Code Review Prompt:

Write your answer / prompt here:

Act as a senior Python engineer. Perform a CRTFC code review (Correctness, Readability, Testability, Failure cases, Compliance with PEP8) on the following function.

Identify:

- Logical errors
- Edge cases
- Security concerns
- PEP8/style issues
- Suggestions for improvement

```
def calculate_discount(cart_items, promo_code):
    """Apply discount to cart total."""
    PROMO_CODES = {
        'SAVE10': 0.10,
        'SAVE20': 0.20,
        'VIP50': 0.50
    }
    total = 0
    for item in cart_items:
        total += item['price'] * item['quantity']
    if promo_code in PROMO_CODES:
        discount = PROMO_CODES[promo_code]
        total = total - discount
    if total < 0:
        total = 0
    return total
```

What did AI find / produce?

Write your answer / prompt here:

Correctness Issue:
The discount is incorrectly subtracted as a fixed value instead of a percentage.

Edge Cases:

- Empty cart_items list
- Missing keys (price, quantity)
- None values
- Invalid promo code

Security Concerns:

- No validation of input structure (could crash or be exploited)

PEP8 Issues:

- Missing spacing
- Constant (PROMO_CODES) should be outside function or uppercase global
- No type hints

Sub-Task B — Bug Investigation Prompt:

Write your answer / prompt here:

Identify the logic bug in the function below.
Then:

1. Explain clearly why it is a bug
2. Provide a corrected version of the function
3. Improve the function with basic input

```
def calculate_discount(cart_items, promo_code):
    PROMO_CODES = {
        'SAVE10': 0.10,
        'SAVE20': 0.20,
        'VIP50': 0.50
    }
    total = 0
    for item in cart_items:
        total += item['price'] * item['quantity']
    if promo_code in PROMO_CODES:
        discount = PROMO_CODES[promo_code]
        total = total - discount
    if total < 0:
        total = 0
    return total
```

Fixed Version

```
def calculate_discount(cart_items, promo_code=None):
    PROMO_CODES = {
        'SAVE10': 0.10,
        'SAVE20': 0.20,
        'VIP50': 0.50
    }

    if not isinstance(cart_items, list):
        raise TypeError("cart_items must be a list")

    total = 0
    for item in cart_items:
        price = item.get('price', 0)
        quantity = item.get('quantity', 0)

        if price < 0 or quantity < 0:
            raise ValueError("Price and quantity must be non-negative")

        total += price * quantity

    if promo_code in PROMO_CODES:
        discount = PROMO_CODES[promo_code]
        total = total * (1 - discount)

    return max(total, 0)
```

What did AI find / produce?

Write your answer / prompt here:

Bug Identified:
total = total - discount subtracts a fixed decimal instead of applying a percentage.

Explanation:
Discount values (e.g., 0.10) represent percentages, so the correct formula should multiply, not subtract directly.

Sub-Task C — Test Generation Prompt:

Write your answer / prompt here:

Generate pytest unit tests for the corrected calculate_discount function.

Cover:

- Happy path (valid cart + valid promo code)
- Empty cart
- Null/None values
- Invalid inputs
- Boundary conditions (zero values, large values)

Use clean pytest structure and meaningful test names.

What did AI find / produce?

Write your answer / prompt here:

1

```
import pytest

def test_happy_path():
    cart = [{'price': 100, 'quantity': 2}]
    assert calculate_discount(cart, 'SAVE10') == 180.0

def test_no_discount():
    cart = [{'price': 50, 'quantity': 1}]
    assert calculate_discount(cart, None) == 50
```

2

```
def test_empty_cart():
    assert calculate_discount([], 'SAVE10') == 0

def test_invalid_cart_type():
    with pytest.raises(TypeError):
        calculate_discount("not a list", 'SAVE10')

def test_negative_values():
    cart = [{'price': -10, 'quantity': 1}]
    with pytest.raises(ValueError):
        calculate_discount(cart, 'SAVE10')
```

3

```
def test_none_values():
    cart = [{'price': None, 'quantity': 1}]
    with pytest.raises(TypeError):
        calculate_discount(cart, 'SAVE10')

def test_large_values():
    cart = [{'price': 1_000_000, 'quantity': 2}]
    assert calculate_discount(cart, 'VIP50')
== 1_000_000
```

■ Reflection — Which sub-task surprised you most and why?

Write your answer / prompt here:

Most impressive: Sub-Task C (Test Generation)

- The AI generated comprehensive and realistic test cases automatically.
- It correctly covered edge cases developers often forget (like invalid types and boundary values).
- The structure followed pytest best practices without needing much correction.

SECTION E | Automation Templates & Personal AI Workflow ■ 3:20 – 4:00

Tasks 11–12 turn everything you practiced today into a reusable personal system. The templates you build here will save you hours every week.

Task 11 Build 3 Personal Reusable Prompt Templates ■ 25 min ● Intermediate

■ **Instructions:**

1. Build ONE reusable prompt template for each of the 3 categories below.
2. Use [PLACEHOLDERS] for the parts that change each time. Include all 5 CRTFC elements.
3. TEMPLATE 1 (General): A recurring communication task from your work (weekly update, client email, meeting request, etc.)
4. TEMPLATE 2 (Dev): A recurring development task (code review, PR description, bug report, test generation, documentation)
5. TEMPLATE 3 (Your choice): Any other task you do regularly that AI could help with
6. For each template: write it, test it by filling in the placeholders, then refine it once.

■ **Deliverable:** 3 complete reusable prompt templates with [PLACEHOLDERS].

■ General — Template 1: General Communication Template

Task this template is for: Weekly team/client update email

5 Keep it concise and professional,,,Use bullet points for clarity,,,,Avoid overly technical jargon (if client-facing)

```
Context: [1] Role: Act as a [2] Task: [3 ] Format: [4] Constraints: [5] Full prompt
(combine above):
```

1 You are communicating project updates to stakeholders.
2 Act as a professional project coordinator.
3 Write a clear and concise weekly update email based on the provided information.
4 Subject line,,Greeting,,,Sections: Progress, Key Updates,Blockers, Next Steps….Professional closing

After testing: what 1 thing did you refine? ~~Added Audience field to~~ better control tone (technical vs non-technical).

■■■ Dev — Template 2: Developer Workflow Template

Task this template is for: Pull Request (PR) description

```
Context: [1] Role: Act as a [2] Task: [3] Format: [4] Constraints: [5] Full prompt
(combine above):
```

1) You are submitting code changes for review.
2) Act as a senior software engineer.
3) Write a clear PR description explaining the changes and impact.
4) Title, Summary, Changes, Testing, Notes
5) Follow best practices, Be concise but informative, Highlight risks and edge cases

After testing: what 1 thing did you refine? ~~Added "Problem being solved"~~ to improve clarity and reviewer understanding.

■ Custom — Template 3: My Choice Template

Task this template is for: Meeting notes → structured summary

```
Context: [ ] Role: Act as a [ ] Task: [ ] Format: [ ] Constraints: [ ] Full prompt
(combine above):
```

1) You have raw, unstructured meeting notes.
2) Act as a professional operations assistant.
3) Convert notes into a structured, professional summary.
4) Attendees, Key Decisions, Action Items (owner + deadline), Blockers
5) Be clear and structured, Do not add assumptions, Keep it concise

After testing: what 1 thing did you refine? ~~Added constraint: "Do not~~ invent missing information" to avoid hallucinated details.

Task 12 Design Your Personal AI Developer Workflow

■ 15 min ● Advanced

■ **Instructions:**

1. List your top 8 most repetitive or time-consuming development and communication tasks.

2. For each, complete the row in the workflow table: tool, prompt type, and estimated time saved per week.

3. Circle your top 3 highest-impact tasks.

4. Prepare to present ONE workflow example to the group in 60 seconds — explain: what the task is, which AI tool you will use, and how much time you expect to save.

■ **Deliverable:** Completed workflow table + 1 example prepared for group sharing.

#	Task Description	Type	AI Tool	Template Name	Est. Time Saved/Week
1	Code review (security, bugs, edge cases)	■ General ■ Dev	Cursor	Code Review Template (CRTFC)	3hrs
2	Bug investigation & debugging	■ General ■ Dev	Cursor	Bug Fix / Investigation Template	2.5hrs
3	Unit test generation (pytest / xUnit)	■ General ■ Dev	Cursor	Test Generation Template	2 hrs
4	PR description writing	■ General ■ Dev	ChatGBT	PR Description Template	1 hr
5	Sprint report summaries	■ General ■ Dev	ChatGBT	Sprint Summary Template	1.5 hr
6	Professional email writing	■ General ■ Dev	Gemini	Email Writing Template	1 hr
7	Meeting notes → structured summary	■ General ■ Dev	Gemini	Meeting Summary Template	1 hr
8	Risk analysis from reports	■ General ■ Dev	ChatGPt	Risk Analysis Template	1.5hr

■ My 60-Second Workflow Pitch (prepare here):

The task I chose: Code review for backend functions (security, bugs, and edge cases)

AI tool I will use: Cursor

The prompt template (summarize): A structured CRTFC prompt where I provide backend system context (e.g., FinTech API), ask the AI to act as a senior backend engineer, review the code for correctness, security vulnerabilities, and edge cases, and return issues by severity with suggested fixes and improved code.

Estimated time saved: 3 hr Impact on my work: Improves code quality, reduces bugs in production, ensures security best practices, and speeds up development workflow.

■ Day 1 — Final Reflection

Complete before the session ends:

The most useful general AI use case I discovered today:

Using AI to quickly generate professional emails and structured summaries from raw notes, saving time and improving clarity.

The most useful developer AI use case I will start this week:

Using AI for code review and bug investigation to identify issues, edge cases, and security risks faster.

One thing I now understand about AI that I did not before:

AI does not truly understand code or language—it predicts outputs based on patterns, so results must always be reviewed.

One question I still have about using AI in my development workflow:

How to safely use AI with sensitive production code without risking data privacy or security exposure?

My commitment: the first AI-assisted task I will do at work tomorrow:

Use AI to review a backend function and generate unit tests to improve code quality and coverage.